

Exp. No: 2

# **SUPPORT VECTOR MACHINE (SVM) AND RANDOM FOREST FOR BINARY & MULTICLASS CLASSIFICATION**

## **Code:**

```
# =====  
# Iris Classification using SVM and Random Forest  
# =====  
  
# 1. Import Libraries  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from matplotlib.colors import ListedColormap  
  
from sklearn.datasets import load_iris  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.svm import SVC  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import accuracy_score,  
ConfusionMatrixDisplay  
  
# =====  
# 2. Load Dataset  
# =====  
  
iris = load_iris()  
  
X = pd.DataFrame(iris.data, columns=iris.feature_names)  
y = iris.target
```

```
print("First Five Rows:")
```

```
print(X.head())
```

```
print("\nDataset Shape:", X.shape)
```

```
# =====
```

```
# 3. Data Visualization
```

```
# =====
```

```
sns.pairplot(  
    X.assign(Species=y),  
    hue="Species",  
    palette="Set2"  
)
```

```
plt.suptitle("Pairwise Relationship Between Iris Features", y=1.02)
```

```
plt.show()
```

```
# =====
```

```
# 4. Split Dataset
```

```
# =====
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.25,  
    random_state=1,  
    stratify=y  
)
```

```
# =====
```

```
# 5. Feature Scaling
```

```
# =====
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Train Models
# =====

svm = SVC(kernel='rbf', random_state=1)
svm.fit(X_train, y_train)

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=1
)

rf.fit(X_train, y_train)

# =====
# 7. Predictions
# =====

svm_pred = svm.predict(X_test)
rf_pred = rf.predict(X_test)

# =====
# 8. Accuracy
# =====

svm_acc = accuracy_score(y_test, svm_pred)
rf_acc = accuracy_score(y_test, rf_pred)

print("\nSVM Accuracy:", round(svm_acc,3))
print("Random Forest Accuracy:", round(rf_acc,3))
```

```
# =====  
# 9. Accuracy Comparison  
# =====
```

```
accuracy = pd.DataFrame({  
    "Model":["SVM","Random Forest"],  
    "Accuracy":[svm_acc, rf_acc]  
})
```

```
plt.figure(figsize=(6,4))
```

```
sns.barplot(  
    data=accuracy,  
    x="Model",  
    y="Accuracy",  
    palette="viridis"  
)
```

```
plt.ylim(0.8,1.02)
```

```
for i,v in enumerate(accuracy["Accuracy"]):  
    plt.text(i,v+0.01,f"{v:.2f}",ha='center',fontsize=11)
```

```
plt.title("Model Accuracy Comparison")  
plt.ylabel("Accuracy")  
plt.show()
```

```
# =====  
# 10. Confusion Matrices  
# =====
```

```
fig,axes=plt.subplots(1,2,figsize=(10,4))
```

```
ConfusionMatrixDisplay.from_predictions(  
    y_test,  
    svm_pred,
```

```
cmap="Blues",
    colorbar=False,
    ax=axes[0]
)
```

```
axes[0].set_title("SVM")
```

```
ConfusionMatrixDisplay.from_predictions(
    y_test,
    rf_pred,
    cmap="Greens",
    colorbar=False,
    ax=axes[1]
)
```

```
axes[1].set_title("Random Forest")
```

```
plt.tight_layout()
plt.show()
```

```
# =====
# 11. Random Forest Feature Importance
# =====
```

```
importance = pd.DataFrame({
    "Feature":iris.feature_names,
    "Importance":rf.feature_importances_
})
```

```
importance = importance.sort_values(
    by="Importance",
    ascending=False
)
```

```
plt.figure(figsize=(7,4))
```

```
sns.barplot(  
    data=importance,  
    x="Importance",  
    y="Feature",  
    palette="mako"  
)
```

```
plt.title("Random Forest Feature Importance")  
plt.xlabel("Importance Score")  
plt.ylabel("Feature")  
plt.show()
```

```
# =====  
# 12. SVM Decision Boundary Visualization  
# (Using first two features)  
# =====
```

```
# Select first two features  
X_vis = X.iloc[:, :2]
```

```
X_train_vis, X_test_vis, y_train_vis, y_test_vis = train_test_split(  
    X_vis,  
    y,  
    test_size=0.25,  
    random_state=1,  
    stratify=y  
)
```

```
# Scale  
scaler_vis = StandardScaler()
```

```
X_train_vis = scaler_vis.fit_transform(X_train_vis)  
X_test_vis = scaler_vis.transform(X_test_vis)
```

```
# Linear SVM for visualization  
svm_vis = SVC(kernel='linear', C=1)
```

```
svm_vis.fit(X_train_vis, y_train_vis)
```

```
# Create mesh grid
```

```
x_min = X_train_vis[:,0].min() - 1
```

```
x_max = X_train_vis[:,0].max() + 1
```

```
y_min = X_train_vis[:,1].min() - 1
```

```
y_max = X_train_vis[:,1].max() + 1
```

```
xx,yy = np.meshgrid(  
    np.arange(x_min,x_max,0.02),  
    np.arange(y_min,y_max,0.02)  
)
```

```
Z = svm_vis.predict(  
    np.c_[xx.ravel(),yy.ravel()]  
)
```

```
Z = Z.reshape(xx.shape)
```

```
# Plot
```

```
plt.figure(figsize=(8,6))
```

```
plt.contourf(  
    xx,  
    yy,  
    Z,  
    alpha=0.35,  
    cmap=ListedColormap([  
        "#FFCCCC",  
        "#CCFFCC",  
        "#CCCCFF"  
    ])  
)
```

```

scatter = plt.scatter(
    X_train_vis[:,0],
    X_train_vis[:,1],
    c=y_train_vis,
    cmap="viridis",
    edgecolor="black",
    s=70
)

# Support vectors
plt.scatter(
    svm_vis.support_vectors_[:,0],
    svm_vis.support_vectors_[:,1],
    s=220,
    facecolors="none",
    edgecolors="red",
    linewidths=2,
    label="Support Vectors"
)

plt.xlabel("Sepal Length (Standardized)")
plt.ylabel("Sepal Width (Standardized)")
plt.title("SVM Decision Regions and Support Vectors")

plt.legend(*scatter.legend_elements(),
           title="Species")

plt.show()

```

## Output:

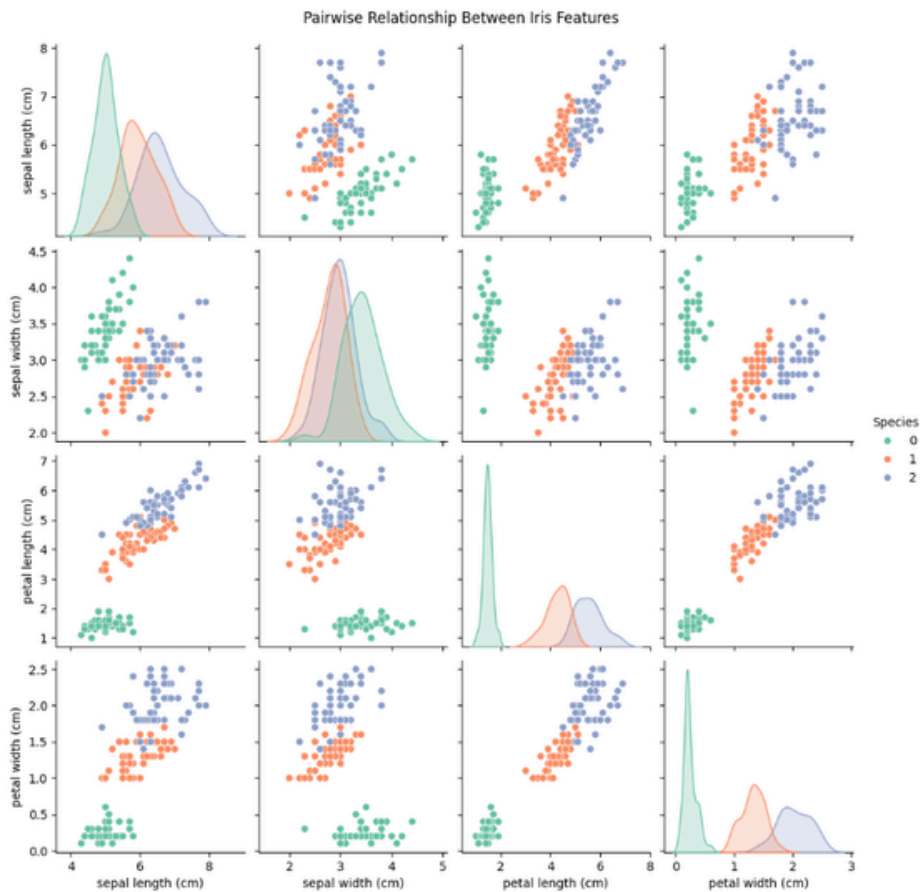
```

First Five Rows:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2

Dataset Shape: (150, 4)

```

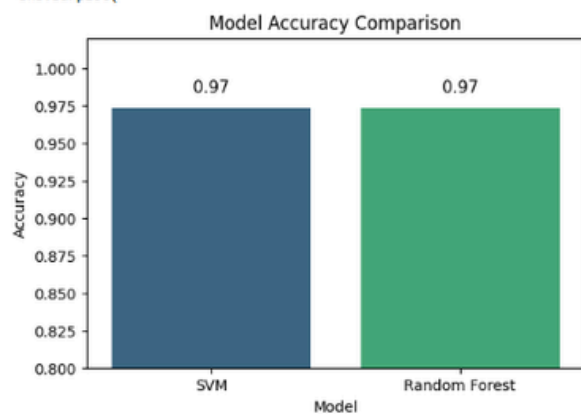


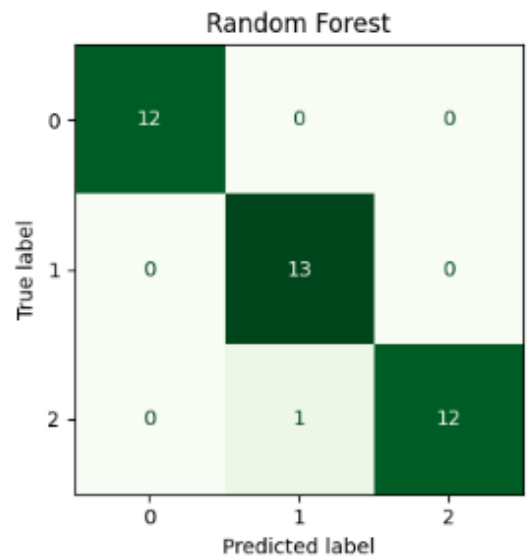
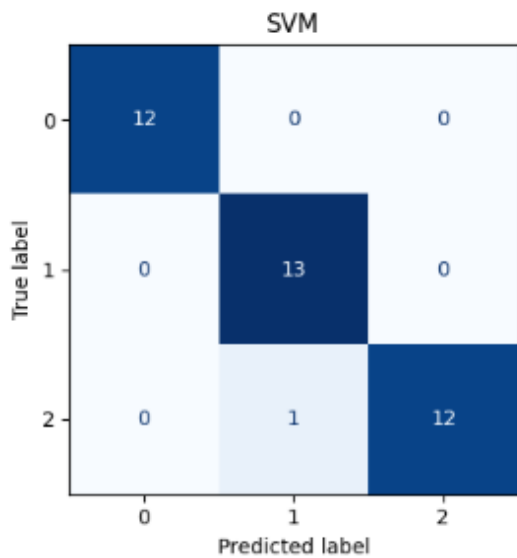


SVM Accuracy: 0.974  
 Random Forest Accuracy: 0.974  
 /tmp/ipykernel\_4173/224648036.py:110: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

sns.barplot(





/tmp/ipykernel\_4173/224648036.py:171: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

sns.barplot(

